AI Assisted Coding

Name: Mohammed Faisaluddin

HT NO: 2303A52214

Batch: 36

Lab 13 – Code Refactoring Using AI (Python) Lab

Objectives

- **Use AI to assist in designing and implementing fundamental data structures in Python**
- **Learn prompt-based structure creation, optimization, and documentation**
- **Improve understanding of Lists, Stacks, Queues, Linked Lists, Graphs, and Hash Tables**
- **Enhance readability and performance with AI-generated suggestions**

Task 1: Removing Code Duplication

```python
def rectangle_details(length, width):
    """
    Calculates and prints the area and perimeter of a rectangle.

    Parameters:
    length (int): Length of the rectangle
    width (int): Width of the rectangle
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)


# Calling function for different rectangles
rectangle_details(5, 10)
rectangle_details(7, 12)
rectangle_details(10, 15)
```

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Explanation

The repeated calculations were moved into a reusable function, reducing duplication and improving readability.

Task 2: Optimizing Loops and Conditionals

```python
import time

names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Converting list to set for faster lookup
name_set = set(names)

start = time.time()

for s in search_names:
    if s in name_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")

end = time.time()

print("Execution Time:", end - start)
```

```
Charlie is in the list
Eve is not in the list
Bob is in the list
Execution Time: 0.0002543926239013672
```

Performance Comparison

Method	Complexity	Description
Nested Loops	$O(n^2)$	Each element compared repeatedly
Set Lookup	$O(n)$	Faster search using hash lookup

Set lookup is faster because sets use hashing.

Task 3: Extracting Reusable Functions

Recommended Algorithms

Operation	Algorithm	Justification
Search by Appointment ID	Hashing	Fast lookup
Sort by Time	Merge Sort	Stable and efficient
Sort by Fee	Merge or Quick Sort	Efficient for large data

```python
def calculate_total(price):
    """
    Calculates total price including tax.

    Parameters:
    price (float): Original price

    Returns:
    float: Total price including tax
    """
    tax_rate = 0.18
    tax = price * tax_rate
    total = price + tax
    return total


print("Total Price:", calculate_total(250))
print("Total Price:", calculate_total(500))
```

```
Total Price: 295.0
Total Price: 590.0
```

Improvement

The repeated tax calculation is now modular and reusable.

Suggest efficient searching and sorting algorithms for an appointment system and implement Python functions with justification.

Task 4: Replacing Hardcoded Values with Constants.

```python
PI = 3.14159
RADIUS = 7

area = PI * (RADIUS ** 2)
circumference = 2 * PI * RADIUS

print("Area of Circle:", area)
print("Circumference of Circle:", circumference)
```

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Improvement

Hardcoded values were replaced with named constants, improving maintainability.

Task 5: Improving Variable Naming

```python
# Base and height of triangle
base = 10
height = 20

# Formula: Area = (base * height) / 2
area_of_triangle = base * height / 2

print("Area of Triangle:", area_of_triangle)
```

```
Area of Triangle: 100.0
```

Improvement

Variables now clearly explain their purpose.

Task 6: Removing Redundant Conditional Logic

```python
def calculate_grade(marks):
    """
    Determines grade based on marks.

    Parameters:
    marks (int): Student marks

    Returns:
    str: Grade
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"


print(calculate_grade(85))
print(calculate_grade(72))
```

```
Grade B
Grade C
```

Improvement

Grading logic is now reusable and centralized.

Task 7: Converting Procedural Code to Functions

```python
def get_input():
    """Gets number from user."""
    return int(input("Enter number: "))


def calculate_square(num):
    """Calculates square of number."""
    return num * num


def display_result(square):
    """Displays the result."""
    print("Square:", square)


number = get_input()
square_value = calculate_square(number)
display_result(square_value)
```

```
Enter number: 665
Square: 442225
```

Improvement

Task 8: Optimizing List Processing

```python
nums = [1, 2, 3, 4, 5]

# Using list comprehension
squares = [n * n for n in nums]

print(squares)
```

```
[1, 4, 9, 16, 25]
```

Improvement

List comprehension makes the code shorter, faster, and more readable.

Final Conclusion

Code refactoring improves readability, maintainability, and performance of legacy code. Using AI-assisted tools helps developers quickly identify code smells such as duplication, inefficient loops, hardcoded values, and poor naming conventions, and transform them into modular and optimized implementations following modern Python best practices.